

Lab 5

This lab will cover content from the fifth week of the course, including:

- Selection
- Trees
- Feature Importance

Learning Objectives

- Interpret decision tree models to infer feature importances
- Compare feature importance information from regression models, PCA, and decision trees
- Visualize data in relevant spaces
- Select and justify use of supervised or unsupervised feature selection methods

Submission And Assignment Instructions

Submit your `Lab5_lastname_firstname.pdf` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ? Questions you must answer or tasks are marked with a " ? "
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ? YOUR CODE HERE`
 - ⭐ Bonus questions are marked with a "⭐ "
 - 🍓 Bonus answers are marked with a "🍓 "

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Data Exploration

1.1: Load the Data

Run the code blocks below to load the dataset and turn the chemical [SMILES strings](#) into interpretable molecular descriptors. This is an example of featurizing (digitalizing)

molecules using [RDKit descriptors](#), which describe the given molecule using one of over 200 molecular properties.

The y-data for these molecules is the hemotoxicity of the molecule.

HemoTox dataset is from [here](#).

```
In [1]: import urllib.request
import ssl
import certifi

# Create a secure SSL context using certifi's bundle
context = ssl.create_default_context(cafile=certifi.where())

# Make your request using the context
url = "https://example.com"
response = urllib.request.urlopen(url, context=context)
html = response.read()
```

```
In [2]: !pip install rdkit
```

```
Collecting rdkit
  Downloading rdkit-2026.3.6-cp313-cp313-manylinux_2_28_x86_64.whl.metadata
(3.8 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packa
ges (from rdkit) (2.1.3)
Requirement already satisfied: Pillow in /usr/local/lib/python3.13/dist-pack
ages (from rdkit) (11.3.0)
Downloading rdkit-2026.3.6-cp313-cp313-manylinux_2_28_x86_64.whl (38.1 MB)
_____ 38.1/38.1 MB 20.3 MB/s eta 0:00:
00
Installing collected packages: rdkit
Successfully installed rdkit-2026.3.6
```

```
In [3]: import numpy as np
import pandas as pd
from rdkit import Chem
from rdkit.Chem import AllChem

#more datasets here: https://github.com/RekerLab/DeepDelta/tree/main/Dataset
url = "https://raw.githubusercontent.com/RekerLab/DeepDelta/main/Datasets/Be
c = pd.read_csv(url)

# look at the top of this dataset
print(c.head())

# convert SMILES to molecules
mols = [Chem.MolFromSmiles(s) for s in c['SMILES']]

# create binary fingerprints for robust ML models
fps = np.array([np.array(AllChem.GetMorganFingerprintAsBitVect(m,2)) for m i

# create descriptors for ML models with higher interpretability
from rdkit.Chem import Descriptors
descr = Descriptors._descList
```

```

calc = [x[1] for x in descr]
d_name = [x[0] for x in descr]

desc = np.array([np.array([f(m) for f in calc]) for m in mols])

# Convert the numpy array to a pandas DataFrame with descriptor names as columns
X = pd.DataFrame(desc, columns=d_name)
# extract y values from original dataframe into separate vector
y = c['Y']
# have a look at the top of this dataframe
print(X.head())
print("Shape of the feature data: ", X.shape)

```

	SMILES	Y
0	<chem>CC(C)(C)c1cc(N)c(SCCN)c(NC(=O)c2cccc(C(=O)Nc3c...</chem>	1.146
1	<chem>C0c1cc(OC)c(C(=O)Nc2cc(C(C)(C)C)cc(N)c2SCCN)cc...</chem>	2.161
2	<chem>CC(C)(C)c1cc(N)c(SCCN)c(NC(=O)c2cc(OCCN)cc(C(=...</chem>	2.037
3	<chem>CC(C)(C)c1cc(N)c(SCCNC(N)=[NH2+])c(NC(=O)c2cc(...</chem>	1.987
4	<chem>NCCSc1c(NC(=O)CCCCNC(N)=[NH2+])cc(C(F)(F)F)cc1...</chem>	1.786

	MaxAbsEStateIndex	MaxEStateIndex	MinAbsEStateIndex	MinEStateIndex	\
0	13.468263	13.468263	0.169002	-0.344563	
1	13.926798	13.926798	0.142263	-0.479838	
2	13.814038	13.814038	0.207461	-0.425490	
3	14.119669	14.119669	0.035630	-0.485341	
4	14.127580	14.127580	0.012675	-4.926127	

	qed	SPS	MolWt	HeavyAtomMolWt	ExactMolWt	\
0	0.119644	11.809524	608.878	564.526	608.296717	
1	0.097933	11.695652	668.930	620.546	668.317846	
2	0.091644	11.760870	667.946	618.554	667.333830	
3	0.025868	11.309091	797.093	738.629	796.421054	
4	0.021686	11.354839	918.996	870.612	918.331760	

	NumValenceElectrons	...	fr_sulfide	fr_sulfonamd	fr_sulfone	\
0	226.0	...	2.0	0.0	0.0	
1	250.0	...	2.0	0.0	0.0	
2	250.0	...	2.0	0.0	0.0	
3	298.0	...	2.0	0.0	0.0	
4	338.0	...	2.0	0.0	0.0	

	fr_term_acetylene	fr_tetrazole	fr_thiazole	fr_thiocyan	fr_thiophene	\
0	0.0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	0.0	

	fr_unbrch_alkane	fr_urea
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	2.0	0.0

[5 rows x 217 columns]

Shape of the feature data: (828, 217)

1.2: Data Exploration

It's important to consider how many observations (molecules) and features you have in any new dataset. With many features over a relatively small set of observations, overfitting is very easy.

? 1.2.1 How many observations and features are in this dataset? Are there missing (NA) observations? Are there any features with 0 variance (only 1 unique value)? Remove missing observations and 0-variance features from your dataset, reporting how many observations and features you removed.

Hint: You can (but don't have to) use `VarianceThreshold` to remove features with 0 variance. If you do it this way, the variance threshold will transform your data into a numpy array and you'll have to use

`selector.get_support(indices=True)` to recover the names of the features kept by the selection and reconstruct a pandas dataframe of your data. You'll need the feature names associated with your dataframe for the plotting in the following question to go smoothly.

```
In [4]: from sklearn.feature_selection import VarianceThreshold
from pandas._libs import missing
## ? YOUR CODE HERE
nObs, nFts = X.shape
print(f"Number of observations: {nObs}")
print(f"Number of features: {nFts}")
missingVals = X.isna().sum()
missingY = y.isna().sum()
print(f"Missing values: {missingVals}")
print(f"Missing values in y: {missingY}")
missingRows = X.isnull().any(axis=1).sum()
print(f"Number of rows with missing values: {missingRows}")
selector = VarianceThreshold(threshold=0.0)
xReduced = selector.fit_transform(X)
mask = selector.get_support()
selectedCol = X.columns[mask]
XFinal = pd.DataFrame(xReduced, columns=selectedCol)
nFts2 = XFinal.shape[1]
print(f"Number of features removed: {nFts - nFts2}")
print(f"Final total number of features: {nFts2}")
```

```
Number of observations: 828
Number of features: 217
Missing values: MaxAbsEStateIndex    0
MaxEStateIndex      0
MinAbsEStateIndex   0
MinEStateIndex      0
qed                  0
..
fr_thiazole         0
fr_thiocyan         0
fr_thiophene        0
fr_unbrch_alkane    0
fr_urea             0
Length: 217, dtype: int64
Missing values in y: 0
Number of rows with missing values: 0
Number of features removed: 27
Final total number of features: 190
```

✓ Answer: The dataset has 828 observations and 217 features. There are no missing values in X or y, so 0 observations were removed. 27 features had zero variance and were removed, leaving 190 features.

We might want to understand how correlated these features are. With so many, they are difficult to visualize individually, but we can make a heatmap to understand whether these molecular descriptors are correlated. These correlations are summary statistics

and may not perfectly show our data, but they can help give us an idea of what the data looks like.

? 1.2.2 Generate a heatmap using `sns.heatmap` to understand how the various molecular descriptors are correlated.

Some requirements:

- Make sure your heatmap has the molecular descriptor feature names on its axes.
- To be able to see all the feature labels, you'll have to make the figure size very large (suggestion: start with `figsize = (42,42)`).
- Use a diverging color scale! The top (1), bottom (-1), and middle (0) of your scale all have meaning, so it's appropriate to visually mark where 0 is.
- Make sure to label your colorbar (what correlation metric is this?)
- Set `annot=False` to prevent the `sns.heatmap` from trying to print the exact correlation on top of every square of the heatmap

Some suggestions:

- set `linewidths=0` for compactness

You may know from background outside the class that it's generally less informative to generate a heat map without clustering and you might be tempted to use `sns.clustermap` . For the purposes of this class, we haven't covered clustering yet, so just use `sns.heatmap` .

```
In [5]: import seaborn as sns
import matplotlib.pyplot as plt

## ? YOUR CODE HERE

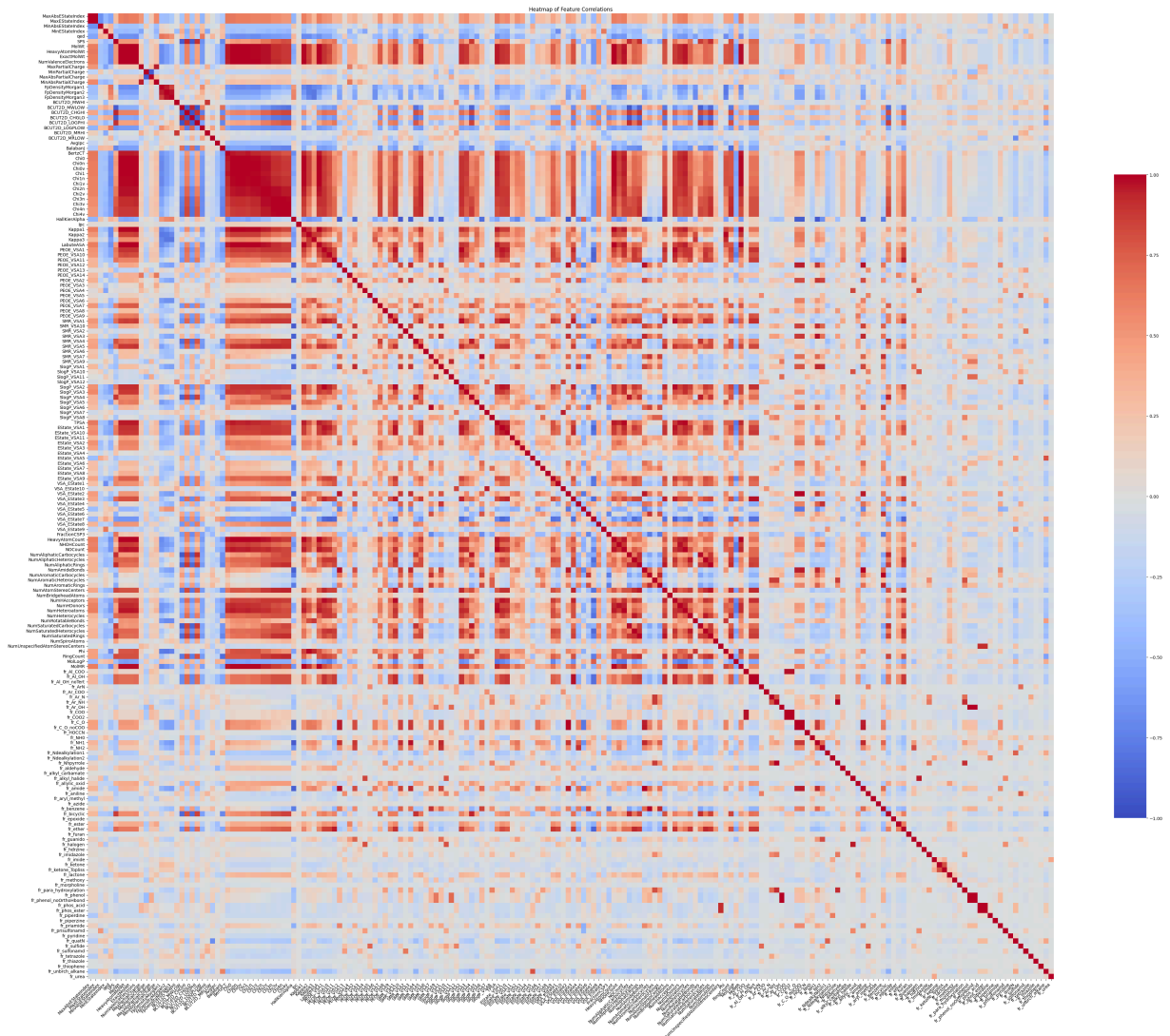
# calculate the Pearson correlation matrix between descriptors
corr = XFinal.corr()

# plot the correlation matrix (using sns.heatmap)
f, ax = plt.subplots(figsize = (42, 42))
sns.heatmap(
    corr,
    cmap = "coolwarm",
    vmin = -1, vmax = 1, center = 0,
    square = True,
    annot = False,
    linewidths = 0,
    xticklabels = True,
    yticklabels = True,
    cbar_kws = {"shrink": 0.5},
    ax = ax
)

# Rotate the tick labels for better readability
```

```
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)

# Display the plot
plt.title('Heatmap of Feature Correlations')
plt.tight_layout()
plt.show()
```



Section 2: PCA

In this section, we're going to visualize the data using PCA (it's generally good practice to visualize your data) and explore feature importances from PCA. PCA identifies orthogonal eigenvectors that represent directions of high variance in the dataset, using the covariance matrix. This matrix assumes our features are all centered on 0, so we will center and re-scale our features before doing PCA.

Each principal component (eigenvector) is a linear combination of the original features, and the coefficients of these combinations are known as loadings. By examining the loadings, one can identify which original features contribute most to the variance

captured by each principal component. Features with high loadings on the first few principal components are usually considered more important, as they contribute significantly to the data's structure, or at least its variance.

When features are highly correlated, PCA tends to capture the variance of these correlated features in fewer principal components. This means that the first few principal components will explain a larger portion of the total variance, as they capture the common variance shared by the correlated features.

? 2.1 Write code below to standardize and scale your data, then do PCA. Visualize the first 3 principal components as an `sns.pairplot`. Label the axes on your pairplot as the percentage variation explained by that principal component. Be sure to adjust `alpha` to avoid overplotting and add a title to your plot.

```
In [6]: from sklearn.preprocessing import StandardScaler
        from sklearn.decomposition import PCA
        import seaborn as sns
        import matplotlib.pyplot as plt

        ## ? YOUR CODE HERE
        # guard against any infinite values, then median-impute
        XClean = XFinal.replace([np.inf, -np.inf], np.nan)
        XClean = XClean.fillna(XClean.median())

        # standardize the features (mean 0, sd 1)
        scaler = StandardScaler()
        XScaled = scaler.fit_transform(XClean)

        # do PCA (named pca, NOT PCA, so the sklearn class isn't overwritten)
        pca = PCA()
        XPCA = pca.fit_transform(XScaled)

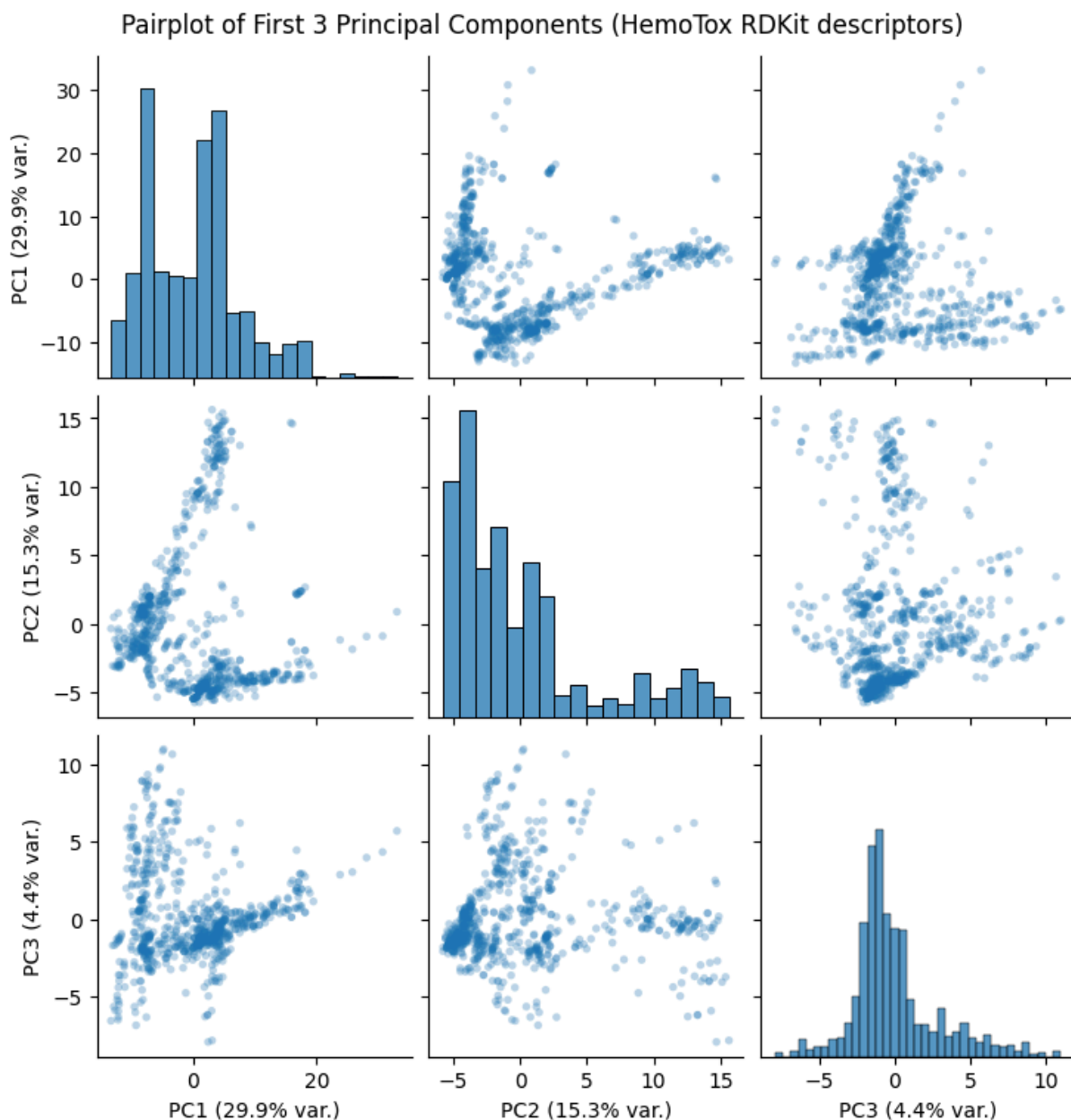
        # get explained variance
        expVar = pca.explained_variance_ratio_
        cumVar = np.cumsum(expVar)
        print(f"PC1: {expVar[0]:.1%}, PC2: {expVar[1]:.1%}, PC3: {expVar[2]:.1%}")
        print(f"PC1-3 together: {cumVar[2]:.1%}")
        print(f"Components needed for 90% variance: {np.argmax(cumVar >= 0.90) + 1}")
        pcLabels = [f"PC{i+1} ({expVar[i]:.1%} var.)" for i in range(3)]
        PCAdf = pd.DataFrame(XPCA[:, :3], columns = pcLabels)

        # pairplot
        g = sns.pairplot(PCAdf, plot_kws = {"alpha": 0.3, "s": 15, "edgecolor": "none"},
                        figure.suptitle("Pairplot of First 3 Principal Components (HemoTox RDKit c
        plt.show())
```

PC1: 29.9%, PC2: 15.3%, PC3: 4.4%

PC1-3 together: 49.6%

Components needed for 90% variance: 33



Looking at the loadings on your principal components, especially components that explain substantial amounts of the variation in your dataset, can be helpful in identifying highly variable features, though these features may also be highly correlated.

? 2.2 Print the top 10 loadings on the first principal component, sorting by absolute value.

```
In [7]: ## ? YOUR CODE HERE

# get the loadings
loadings = pd.DataFrame(
    pca.components_[:2].T,
    columns = ["PC1", "PC2"],
    index = XClean.columns
)
```

```
# sort and display the loadings
for PC in ["PC1", "PC2"]:
    top = loadings[PC].reindex(loadings[PC].abs().sort_values(ascending = False))
    print(f"\nTop 10 features driving {PC}:")
    print(top.head(10).round(3))
```

Top 10 features driving PC1:

Chi2n	0.131
Chi2v	0.131
Chi3v	0.129
Chi3n	0.129
NumValenceElectrons	0.129
ExactMolWt	0.128
MolWt	0.128
HeavyAtomMolWt	0.128
Chi1v	0.128
Chi1n	0.127

Name: PC1, dtype: float64

Top 10 features driving PC2:

fr_NH1	0.172
SMR_VSA3	0.169
HallKierAlpha	-0.169
SMR_VSA10	0.168
NumAmideBonds	0.167
fr_amide	0.167
SlogP_VSA1	0.167
PEOE_VSA12	0.161
VSA_EState2	0.159
fr_C_0_noC00	0.158

Name: PC2, dtype: float64

Section 3: Regularized Multiple Linear Regression - the LASSO

Unlike Ridge regression or simple multiple linear regression, L1 regularization (the LASSO) can set coefficients of features to 0, effectively selecting a subset of features as "important" for the model.

? 3.1 Use the start of the code below to build a simple linear regression model to predict the output, using L1 regularization (LASSO). How many coefficients does this model set equal to 0? Print the top 10 coefficients (top by absolute value). A test-train split has been established for you. Initialize the LASSO model with `alpha = 0.1`, `random_state = 42`.

```
In [8]: from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```

# Split the data into training and testing sets
XTrain, XTest, yTrain, yTest = train_test_split(XScaled, y, test_size = 0.2,

## ? YOUR CODE HERE

# Initialize the Lasso model (parameters specified by the assignment)
lasso = Lasso(alpha = 0.1, random_state = 42, max_iter = 10000)

# fit the model on the training data
lasso.fit(XTrain, yTrain)

# Get the coefficients and their corresponding feature names
coefs = pd.Series(lasso.coef_, index = XClean.columns)
print(f"Coefficients set to 0: {(coefs == 0).sum()} of {len(coefs)}")
print(f"Non-zero coefficients: {(coefs != 0).sum()}")

# sort by absolute value
topCoefs = coefs.reindex(coefs.abs().sort_values(ascending = False).index)

# Print the top 10 features with the largest |coefficient|
print("\nTop 10 features by |coefficient|:")
print(topCoefs.head(10))

```

Coefficients set to 0: 180 of 190

Non-zero coefficients: 10

Top 10 features by |coefficient|:

NumSpiroAtoms	-0.135731
FpDensityMorgan1	0.110947
BCUT2D_CHGHI	-0.109317
SMR_VSA5	-0.102253
fr_lactone	-0.096532
BalabanJ	0.075054
PEOE_VSA10	-0.027616
VSA_EState8	-0.007691
VSA_EState9	0.005467
fr_Ndealkylation1	0.001380

dtype: float64

How exciting! Our LASSO discarded a lot of features that it didn't find to be important. It could be that hemotoxicity is a function of just a few molecular features! Before we can conclude that, we should check to see if this model is actually a good fit for the data.

? 3.2 Does this model fit the data well? Make predictions on the test set and calculate the coefficient of determination (R^2) between the test predictions and test data. (1 sentence answer). Should you use this model to interpret feature importances? (1 additional sentence)

We'll revisit the LASSO to run cross-validation and compare to some decision trees later. This is just a first-pass check on one test data set.

In [9]: `## ? YOUR CODE HERE`
`# Predict on the test set using the fitted model`

```
yPred = lasso.predict(XTest)

# Calculate the R^2 value and print
r2 = r2_score(yTest, yPred)
print(f"R^2 value of the LASSO model on test data: {r2:.4f}")
```

R² value of the LASSO model on test data: 0.4946

✓ Answer: No, the LASSO explains only about half the variance in the test set (test $R^2 = 0.49$), so it's a medium fit. We shouldn't lean on it for feature importances and with so many highly correlated descriptors, LASSO arbitrarily keeps one from each correlated group and zeroes the rest, so a "zero" coefficient doesn't mean a feature is unimportant.

Section 4: Decision Trees

? 4.1 Write code below to build and evaluate a simple decision tree on all the data, using 5-fold cross-validation. Use `random_state = 12` to seed the Decision tree. Print the RMSE plus or minus its standard error (not standard deviation) from the cross-validation.

```
In [10]: from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_selection import VarianceThreshold
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

## ? YOUR CODE HERE

# Initialize and train the decision tree regressor using 5-fold cross-validation
model = Pipeline([("scaler", StandardScaler()),
                  ("tree", DecisionTreeRegressor(random_state = 12))
])

# run cv on the full model
mseScores = cross_val_score(model,
                             XClean,
                             y,
                             cv = 5,
                             scoring = "neg_mean_squared_error")
rmseScores = np.sqrt(-mseScores)
rmseMean = np.mean(rmseScores)
rmseSE = np.std(rmseScores, ddof = 1) / np.sqrt(len(rmseScores))

# print RMSE (nice printing example)
print(f"Cross-validated RMSE (full model): {rmseMean:.5f} ± {rmseSE:.5f}")
```

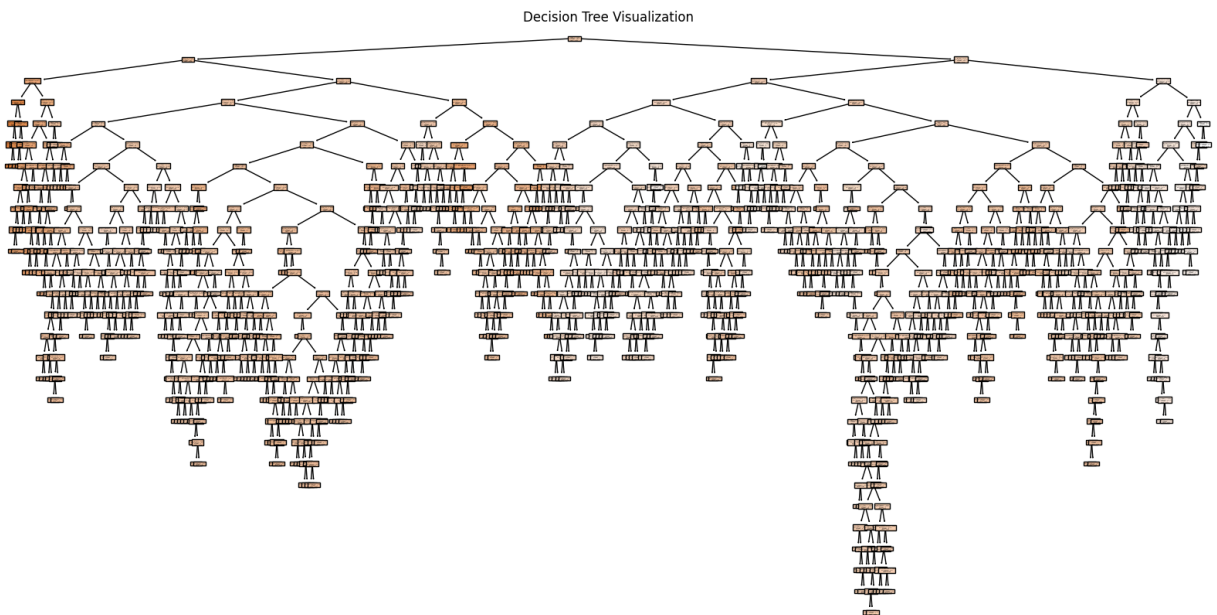
Cross-validated RMSE (full model): 0.79969 ± 0.04645

If we try to visualize this model (using the code block below, which assumes your scaled dataframe is called `X_scaled` and refits the model to the entire dataset), you'll see that the very large number of features has made this model difficult to interpret, even if

the tree organizes more important features to the top. (This is not supposed to be a readable figure, it is here to demonstrate the complexity of the tree growth from these data).

```
In [11]: # Train the model on the entire dataset
model.fit(XClean, y)
modelFitted = model.named_steps["tree"]

# Visualize the decision tree
plt.figure(figsize = (20, 10))
plot_tree(modelFitted, feature_names = XClean.columns, filled = True)
plt.title("Decision Tree Visualization")
plt.show()
```



? 4.2 Build a second model, this time with all the features but a reduced depth (use `random_state = 12, max_depth = 4`). Report the 5-fold cross-validation RMSE, plus or minus its standard error (not standard deviation). *Start naming your models and RMSEs different things, you're going to have to compare them later.*

```
In [12]: ## ? YOUR CODE HERE
# Build a model with a limited depth
modelShallow = Pipeline([("scaler", StandardScaler()),
    ("tree", DecisionTreeRegressor(max_depth = 4, random_state = 12))
])

# cv on the shallow model, using all the data
mseScoresShallow = cross_val_score(modelShallow,
    XClean,
    y,
    cv = 5,
    scoring = "neg_mean_squared_error")

rmseScoresShallow = np.sqrt(-mseScoresShallow)
rmseMeanShallow = np.mean(rmseScoresShallow)
rmseSEShallow = np.std(rmseScoresShallow, ddof=1) / np.sqrt(len(rmseScoresShallow))
```


? 4.4 Build a model with a reduced input feature set, using a Variance Threshold of 0.2 to remove low-variance features. Continue limiting the depth of the tree with `random_state = 12, max_depth = 4`. Report the 5-fold cross-validation RMSE, plus or minus its standard error (not standard deviation).

you can use `selected_features = selector.get_support(indices = True)` to get the feature names of the of the data left after selection.

```
In [14]: ## ? YOUR CODE HERE
from sklearn.impute import SimpleImputer

# Build a second model with low-variance features removed
modelFiltered = Pipeline([
    ("vt", VarianceThreshold(threshold = 0.2)),
    ("scaler", StandardScaler()),
    ("tree", DecisionTreeRegressor(max_depth = 4, random_state = 12))
])

# Initialize and train the reduced model using 5-fold cross-validation
mseScoresFilt = cross_val_score(modelFiltered, XClean, y, cv = 5, scoring = "neg_mean_squared_error")
rmseScoresFilt = np.sqrt(-mseScoresFilt)
rmseMeanFilt = np.mean(rmseScoresFilt)
rmseSEFilt = np.std(rmseScoresFilt, ddof = 1) / np.sqrt(len(rmseScoresFilt))
print(f"5-fold CV RMSE (filtered shallow tree): {rmseMeanFilt:.5f} ± {rmseSEFilt:.5f} (SE)")

# cv on the reduced model, using all the data
modelFiltered.fit(XClean, y)
selected_features = modelFiltered.named_steps["vt"].get_support(indices = True)
print(f"Features kept: {len(selected_features)} of {XClean.shape[1]}")
```

5-fold CV RMSE (filtered shallow tree): 0.70795 ± 0.04629 (SE)
Features kept: 134 of 190

? 4.5 Use 5-fold cross-validation to compute RMSE and evaluate your LASSO model from earlier. Report the 5-fold cross-validation RMSE, plus or minus its standard error (not standard deviation).

```
In [15]: ## ? YOUR CODE HERE
# cross-validation, lasso is already defined above
mseScoresLasso = cross_val_score(lasso, XScaled, y, cv = 5, scoring = "neg_mean_squared_error")
rmseScoresLasso = np.sqrt(-mseScoresLasso)
rmseMeanLasso = np.mean(rmseScoresLasso)
rmseSELasso = np.std(rmseScoresLasso, ddof = 1) / np.sqrt(len(rmseScoresLasso))
print(f"5-fold CV RMSE (LASSO): {rmseMeanLasso:.5f} ± {rmseSELasso:.5f} (SE)")
```

5-fold CV RMSE (LASSO): 0.68967 ± 0.11747 (SE)

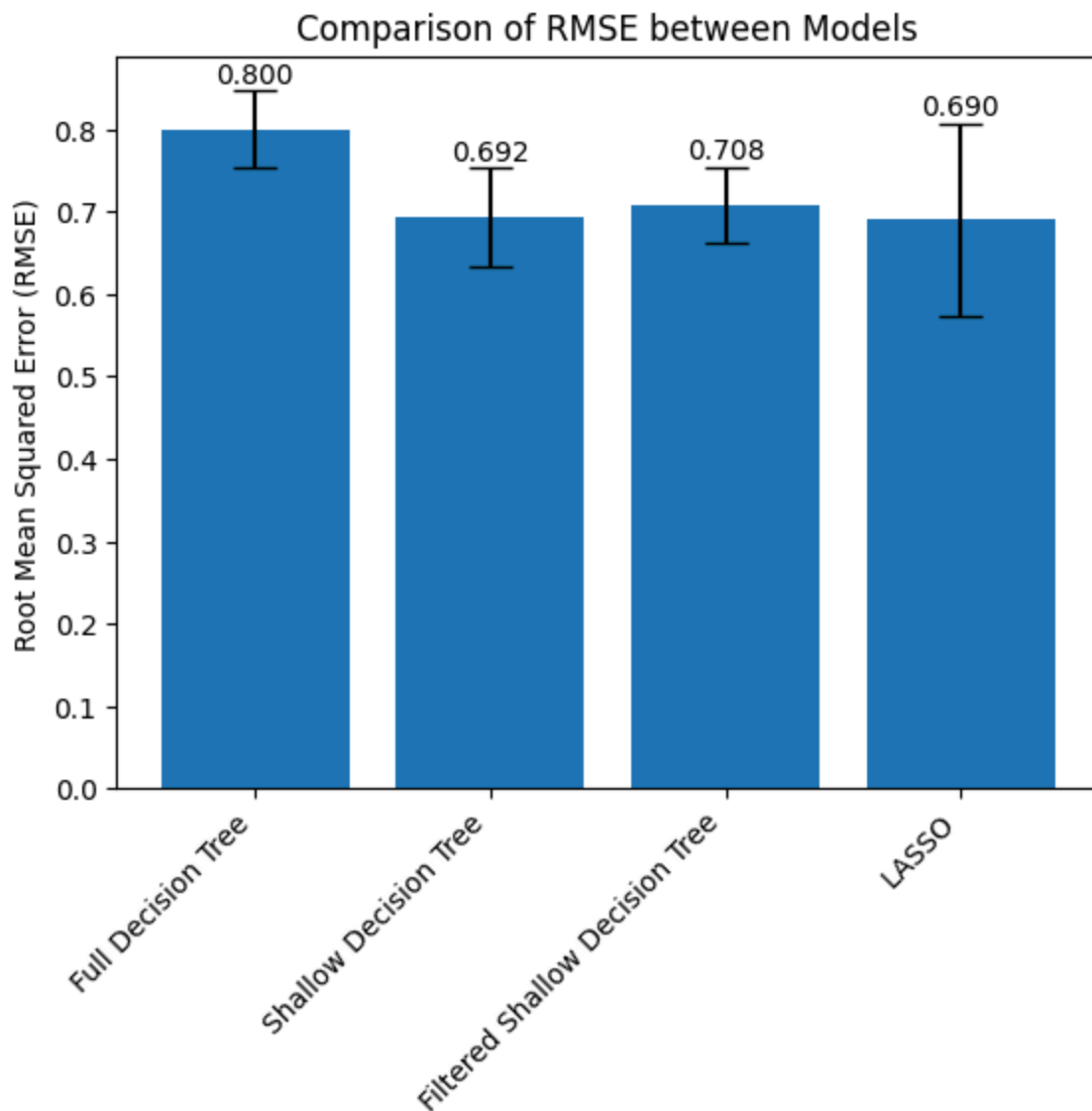
? 4.6 Plot and compare all 4 models' RMSE with error bars (standard error).

Your models should be the "Full Decision Tree", "Shallow Decision Tree", "Filtered Shallow Decision Tree" and "LASSO"

```
In [16]: ## ? YOUR CODE HERE
# Compare the models' RMSEs with error bars
modelName = ["Full Decision Tree", "Shallow Decision Tree", "Filtered Shallow Decision Tree", "LASSO"]
rmseMeans = [rmseMean, rmseMeanShallow, rmseMeanFilt, rmseMeanLasso]
rmseSEs = [rmseSE, rmseSEShallow, rmseSEFilt, rmseSELasso]

plt.bar(modelName, rmseMeans, yerr = rmseSEs, capsize = 8)
for i, (m, s) in enumerate(zip(rmseMeans, rmseSEs)):
    plt.text(i, m + s + 0.01, f"{m:.3f}", ha = "center")

plt.ylabel('Root Mean Squared Error (RMSE)')
plt.title('Comparison of RMSE between Models')
plt.xticks(rotation = 45, ha = 'right')
plt.show()
```



? 4.7 Was there a significant change in model performance when you compare using all the data to using just a subset of the data (with the same `max_depth = 4` parameter)? Why or why not? *Hint: Both of these limited-depth trees have used "early*

stopping" to avoid overfitting. Generally, what variables are these trees using to make decisions?

You should not have to do a statistical test to see whether or not there was a significant difference between these two conditions, it should be obvious from looking at your plot.

✓ Answer: No. The filtered shallow tree (RMSE = 0.71 ± 0.05) and the shallow tree on all features (= 0.69 ± 0.06) have heavily overlapping error bars. A depth-4 tree makes at most 15 splits, and it picks the most informative features (here Chi3n carries about 43% of the importance, followed by MolMR), so getting rid of low-variance features doesn't matter because the tree was mostly ignoring them anyway.

Variance thresholding is unsupervised as it knows nothing about y , and on unscaled descriptors it's scale-dependent, so it can drop a useful feature. For example, fr_lactone (2nd most important in the unfiltered tree) gets removed at 0.2.

? 4.8 Were any of these good models? Since RMSE has the same units as the response variable (like standard deviation does), compare the RMSE to the scale of the data by making a histogram and/or boxplot of the response variable to understand if RMSE around 0.7 is reasonable, given the scale of the data. *Feel free to explicitly calculate standard deviation, IQR, or other summary metrics that help you understand this data distribution. A plot of the distribution of the response variable is a much more complete qualitative picture than just summary metrics, which is why it's provided here as a suggestion as a place to start.*

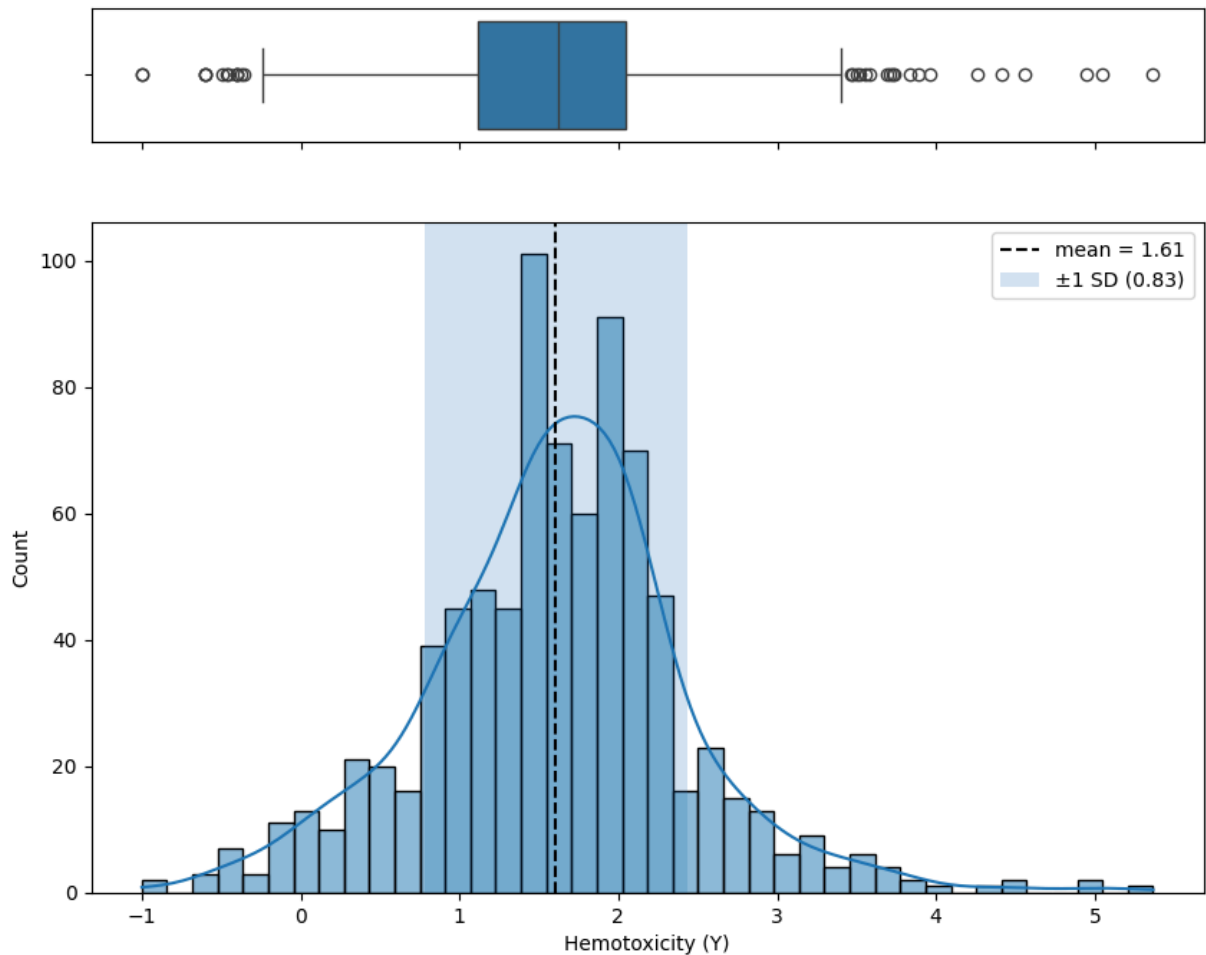
```
In [17]: ## ? YOUR CODE HERE
# distribution of the hemotoxicity to judge RMSE reasonability
ySD = y.std()
yIQR = y.quantile(0.75) - y.quantile(0.25)
print(f"mean = {y.mean():.3f}, SD = {ySD:.3f}, IQR = {yIQR:.3f}, range = [{y
print(f"Best CV RMSE / SD = {min(rmseMeanShallow, rmseMeanLasso) / ySD:.2f}")

fig, (ax_box, ax_hist) = plt.subplots(nrows = 2, sharex = True,
                                     gridspec_kw = {"height_ratios": [1, 5]},
                                     figsize = (10, 8))

sns.boxplot(x = y, ax = ax_box)
ax_box.set_xlabel("")
sns.histplot(y, bins = 40, kde = True, ax = ax_hist)
ax_hist.axvline(y.mean(), color = "k", linestyle = "--", label = f"mean = {y
ax_hist.axvspan(y.mean() - ySD, y.mean() + ySD, alpha = 0.2, label = f"±1 SD
ax_hist.set_xlabel("Hemotoxicity (Y)")
ax_hist.set_ylabel("Count")
ax_hist.legend()
fig.suptitle("Distribution of Hemotoxicity Response")
plt.show()
```

```
mean = 1.605, SD = 0.826, IQR = 0.929, range = [-1.00, 5.37]
Best CV RMSE / SD = 0.84
```

Distribution of Hemotoxicity Response



✅ Answer (~1 sentence): Not really because Y has an SD of about 0.83 (IQR = 0.93), and a model that just predicts the mean would have an RMSE of about 0.83, so an RMSE of about 0.69 is only about 17% better than that baseline (CV $R^2 = 0.3$). The models capture some signal but predict weakly.

Section 5: Math - Node impurities

In the regression case, decision trees can minimize mean squared error between the predicted response value and the true response value. In the classification case, the "error" is slightly more complicated.

We're going to calculate some node impurities by hand! **Please show all your work for this section. Submit typeset or clear handwritten work.**

We're going to look at a toy dataset of apples and bananas, each of which has an associated color and weight. In this scenario, we're trying to predict the type of fruit based on its properties.

Weight (g)	Color	Type
155	green	Apple
132	red	Apple
175	yellow	Banana
152	green	Banana
141	yellow	Apple
170	yellow	Banana

? 5.1 Calculate the gini index (also called impurity) of the root node (all the samples).

✓ Answer: $Gini = 1 - \sum_k p_k^2$. The root has 3 apples and 3 bananas, so
 $p_A = p_B = 3/6 = 0.5$.

$$Gini_{root} = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = 0.5$$

? 5.2 Split the fruits by whether or not their weight is greater than 150 g and calculate the Gini index for both resulting nodes.

✓ Answer:

Node 1 (≤ 150 g): {132 Apple, 141 Apple} $\rightarrow$ 2 apples, 0 bananas.

$$Gini = 1 - (1^2 + 0^2) = 0 \text{ (pure)}$$

Node 2 (> 150 g): {155 Apple, 175 Banana, 152 Banana, 170 Banana} $\rightarrow$ 1 apple, 3 bananas. $Gini = 1 - ((1/4)^2 + (3/4)^2) = 1 - (1/16 + 9/16) = 6/16 = 0.375$

$$\text{Weighted: } \frac{2}{6}(0) + \frac{4}{6}(0.375) = 0.25$$

? 5.3 Split the fruits by whether or not they're yellow and calculate the Gini index for both resulting nodes.

✓ Answer:

Node 1 (yellow): {175 Banana, 141 Apple, 170 Banana} $\rightarrow$ 1 apple, 2 bananas. $Gini = 1 - ((1/3)^2 + (2/3)^2) = 1 - 5/9 = 4/9$

Node 2 (non-yellow): {155 Apple (green), 132 Apple (red), 152 Banana (green)} $\rightarrow$ 2 apples, 1 banana. $Gini = 1 - ((2/3)^2 + (1/3)^2) = 4/9$

$$\text{Weighted: } \frac{3}{6}(4/9) + \frac{3}{6}(4/9) = 4/9$$

? 5.4 Which split results in the greatest weighted average decrease in impurity?

✓ Answer: The weight > 150 g split. Its decrease is $0.5 - 0.25 = 0.25$, compared with $0.5 - (4/9) = 0.056$ for the yellow split. The weight split produces one pure node and

the color split leaves both children nearly as mixed as the root.